



廈門大學

《操作系统》实验三

姓名: 李煜瑒
学号: 37220232203725
年级: 2023
专业: 人工智能
日期: 2025.11.28

目录

一、实验目的.....	3
二、实验任务.....	3
三、任务实现.....	3
3.1 调度算法实现	3
3.1.1 默认调度算法（随机选择）	3
3.1.2 轮转调度算法（RR）	4
3.1.3 优先级调度算法（Priority）	5
3.1.4 先来先服务调度算法（FCFS）	5
3.1.5 静态多级队列调度算法（SML）	6
3.2 测试程序实现	7
3.2.1 工作函数定义	8
3.2.2 测试1	9
3.2.3 测试2	9
3.2.4 测试3	9
四、测试与结果.....	9
4.1 测试1	10
4.2 测试2	10
4.3 测试3	11
五、结果分析.....	11
5.1 各调度算法特性分析.....	11
(1) DEFAULT（随机调度）	12
(2) RR（轮转调度）	12
(3) PRIORITY（优先级调度）	12
(4) FCFS（先来先服务）	13
(5) SML（静态多级队列）	13
5.2 核心问题分析	14
(1) 车队效应（Convoy Effect）	14
(2) 响应性（Responsiveness）	14
(3) 饥饿（Starvation）	15
5.3 算法对比总结	15
六、实验总结.....	15
6.1 实验收获.....	15
6.2 各算法适用场景	16
6.3 改进方向.....	16
6.4 实验结论.....	17
附录.....	17
1 测试结果与测试数据.....	17
场景1：纯CPU长短作业（车队效应）	17
场景2：交互型 vs 背景任务.....	19
场景3：优先级饥饿测试	21
2 关键代码	23

一、实验目的

1. 深入理解操作系统中进程调度的基本原理和重要性
2. 掌握多种经典调度算法的设计思想和实现方法：
 - 默认调度（随机选择）
 - 轮转调度（Round Robin, RR）
 - 优先级调度（Priority Scheduling）
 - 先来先服务调度（First Come First Served, FCFS）
 - 静态多级队列调度（Static Multi-Level Queue, SML）
3. 通过实现上述调度算法，加深对操作系统内核的理解
4. 设计合理的测试场景，定量分析不同调度算法的性能特点和适用场景
5. 理解调度算法在公平性、响应性、吞吐量等方面的权衡

二、实验任务

本实验需要完成以下任务：

【任务 1】在内核中实现五种调度算法

- 修改 `proc.c` 文件，实现各调度算法的选择逻辑
- 实现 `setscheduler()` 系统调用，支持运行时切换调度算法
- 确保各调度算法能正确选择下一个运行的进程

【任务 2】设计调度算法测试程序

- 编写 `scheduler_test.c` 用户程序
- 设计三个典型测试场景，突出不同调度算法的差异
- 收集进程的就绪时间、运行时间、睡眠时间、周转时间等统计数据

三、任务实现

3.1 调度算法实现

所有调度算法实现位于 `proc.c` 文件中，通过函数指针实现调度策略的切换。

3.1.1 默认调度算法（随机选择）

实现思路：从随机位置开始扫描进程表，选择第一个可运行进程。

核心代码：

```

struct proc * defaultScheduler(void) {
    struct proc * p;
    int i, rnd;

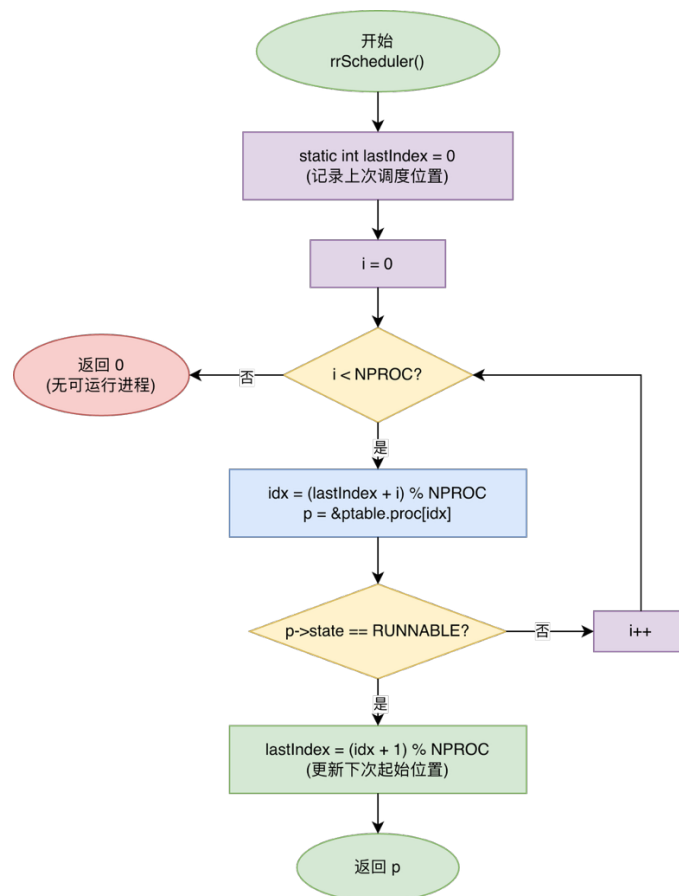
    rnd = random(NPROC);
    for (i = 0; i < NPROC; i++) {
        p = ptable.proc + ((rnd + i) % NPROC);
        if (p->state == RUNNABLE)
            return p;
    }
    return 0;
}

```

随机选择算法实现简单，但调度行为不可预测，无法保证公平性。

3.1.2 轮转调度算法（RR）

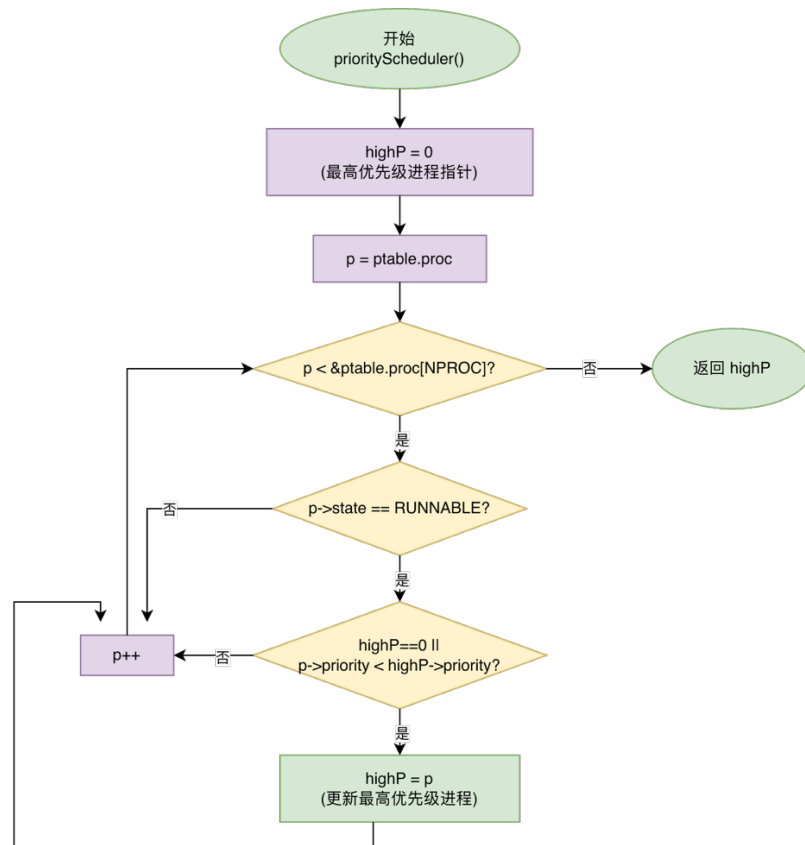
实现思路： 记录上一次调度的位置，从该位置向后轮转选择下一个可运行进程。



轮转算法可以让所有进程公平地获得 CPU 处理的机会，不识别优先级，是最公平的算法，可以比较好的避免某个进程垄断一整个 CPU。

3.1.3 优先级调度算法（Priority）

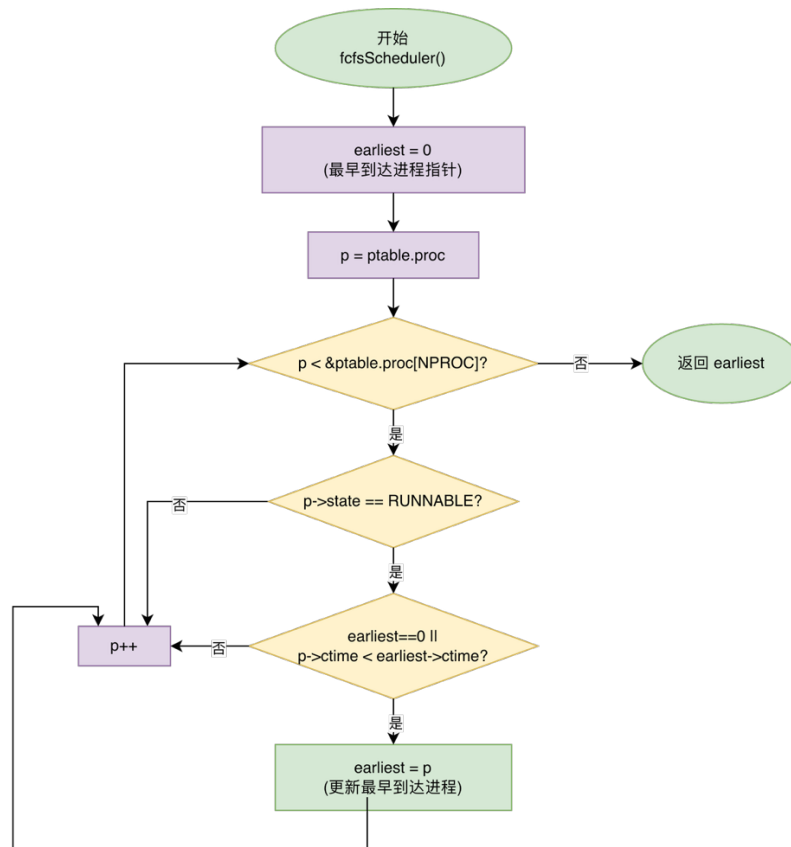
实现思路： 遍历进程表，选择优先级数值最小（即优先级最高）的可运行进程。



优先级调度算法会让优先级较高的进程先获得 CPU，适合实时系统等对优先级要求高的场景。

3.1.4 先来先服务调度算法（FCFS）

实现思路： 遍历进程表，选择创建时间最早的可运行进程。



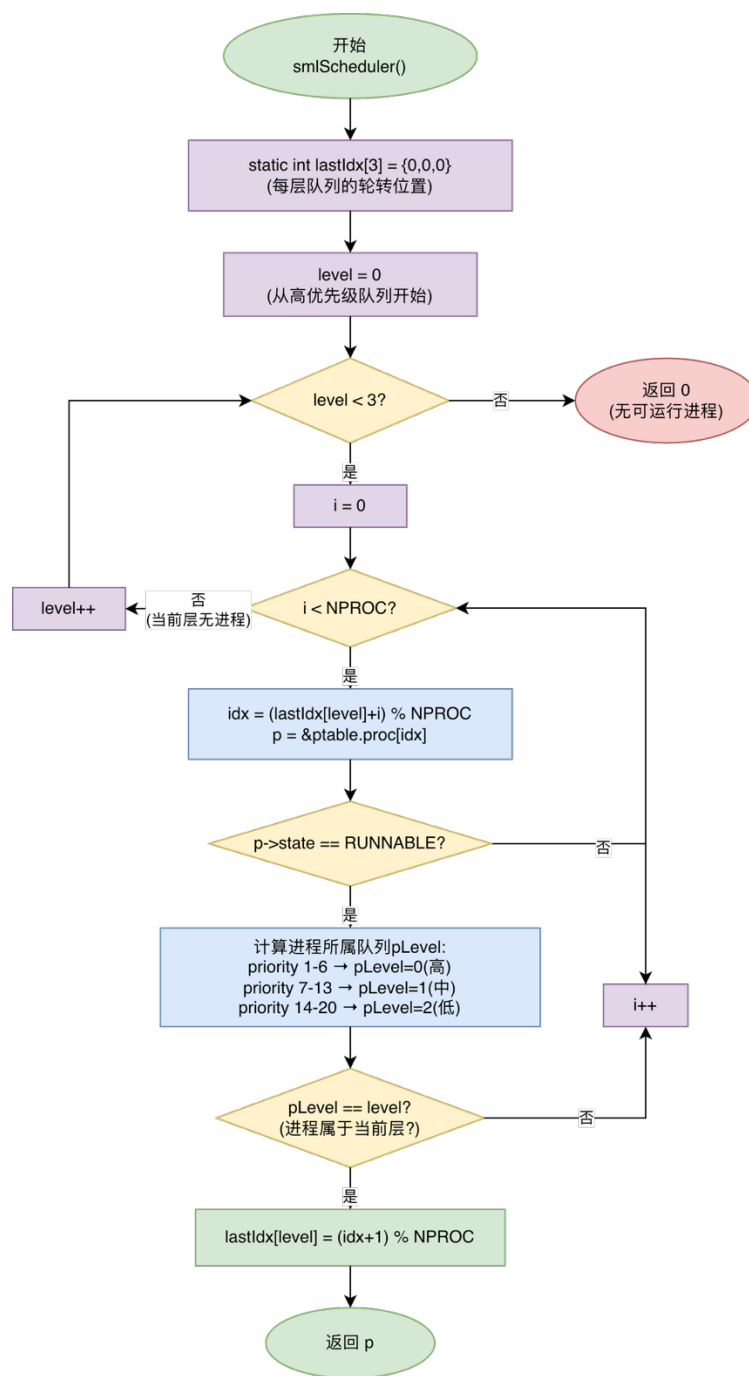
先来先服务算法按照顺序进行调度，对短作业可能不够友好，容易产生车队效应。

3.1.5 静态多级队列调度算法（SML）

实现思路： 按优先级将进程分为三个队列层级，优先调度高层队列；同层内采用轮转。

优先级分层：

- 高层：优先级 1–6
- 中层：优先级 7–13
- 低层：优先级 14–20



静态多级队列调度算法结合了优先级调度和轮转调度的优点，既可以保证高优先级任务的快速响应，也可以保证公平性。

3.2 测试程序实现

在所有调度算法的对比测试中，每次测试固定创建 6 个子进程，覆盖 CPU 密集型、IO 密集型以及交互型等不同类型的工作负载。程序通过扩展系统调用 `wait2()` 记录每个进程的就绪时间（`retime`）、运行时间（`runtime`）、睡眠时间（`stime`），并以 **周转时间 = `retime` + `runtime` + `stime`** 进行统计。

- 平均就绪时间：衡量进程从创建到真正获得 CPU 的等待时长，反映响应性和是否饥饿。
- 平均运行时间：反映进程实际占用 CPU 的时间，主要由工作量决定。
- 平均睡眠时间：主要由 `sleep` 和 IO 决定，配合就绪时间一起分析交互型任务是否能及时被唤

醒并继续执行。

- 平均周转时间：从创建到完成的总时间。

3.2.1 工作函数定义

```
#define CPU_WORK 500000 // CPU 基准工作量
#define IO_WORK 20 // IO 基准次数
```

CPU 密集型工作：

```
void cpu_work(int work) {
    int i, j;
    volatile int sum = 0; // volatile 防止编译器优化
    for (i = 0; i < work; i++) {
        for (j = 0; j < 100; j++) {
            int x = i * j + (i ^ j);
            x = (x << 1) ^ (x >> 1);
            sum += x;
        }
    }
}
```

IO 密集型工作：

```
void io_work(int rounds) {
    int i, fd;
    char buf[] = "io test\n";
    for (i = 0; i < rounds; i++) {
        fd = open("io_test", O_CREATE | O_WRONLY);
        if (fd >= 0) {
            write(fd, buf, sizeof(buf) - 1);
            close(fd);
        }
        sleep(15);
    }
}
```

交互型工作：

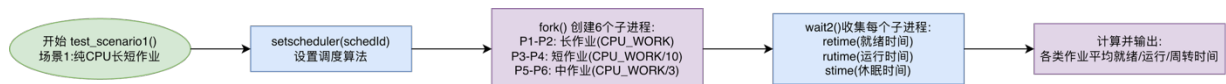
```
void interactive_work(int rounds) {
    int i;
    for (i = 0; i < rounds; i++) {
        cpu_work(CPU_WORK / 50);
        sleep(10);
    }
}
```


3.2.2 测试1

设计目的：测试不同的调度算法是否会出现长作业阻塞短作业的情况。

进程设计：

- 2 个长作业：CPU_WORK 工作量，优先级 10，最先创建
- 2 个短作业：CPU_WORK/10 工作量，优先级 10，sleep(10) 后创建
- 2 个中作业：CPU_WORK/3 工作量，优先级 10，再 sleep(10) 后创建

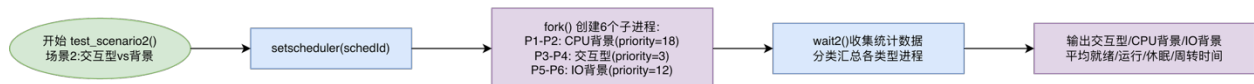


3.2.3 测试2

设计目的：测试调度算法对交互型任务的响应能力。

进程设计：

- 2 个 CPU 任务：CPU_WORK 工作量，优先级 18（低）
- 2 个交互任务：短 CPU + 频繁 sleep，优先级 3（高）
- 2 个 IO 任务：磁盘写入 + sleep，优先级 12（中）

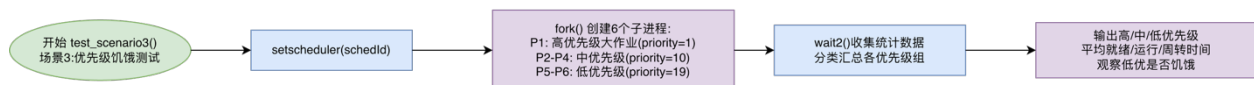


3.2.4 测试3

设计目的：测试调度算法是否会导致低优先级任务饥饿。

进程设计：

- 1 个高优任务：CPU_WORK*2 工作量，优先级 1
- 3 个中优任务：CPU_WORK/2 工作量，优先级 10
- 2 个低优任务：CPU_WORK/2 工作量，优先级 19



四、测试与结果

4.1 测试1

调度算法	长作业平均就绪	长作业平均周转	短作业平均就绪	短作业平均周转	中作业平均就绪	中作业平均周转
DEFAULT	2	25	16	17	11	17
RR	7	42	2	4	7	12
PRIORITY	1	23	16	18	14	20
FCFS	1	26	7	8	10	31
SML	4	30	2	4	3	7

- RR 和 SML：短作业周转时间最短（4 ticks），就绪等待最短（2 ticks）
- DEFAULT / PRIORITY：短作业周转 17–18 ticks，存在车队效应

4.2 测试2

调度算法	进程类型	就绪时间	运行时间	周转时间
DEFAULT	交互型	54	6	249
DEFAULT	CPU	3	27	30
DEFAULT	IO	74	31	577
RR	交互型	24	3	241
RR	CPU	10	31	41
RR	IO	44	41	664
PRIORITY	交互型	13	4	213
PRIORITY	CPU	4	25	29
PRIORITY	IO	39	23	704
FCFS	交互型	41	20	263
FCFS	CPU	4	26	31
FCFS	IO	82	28	613
SML	交互型	10	3	213
SML	CPU	16	18	34
SML	IO	31	56	679

- PRIORITY 和 SML：交互型就绪时间最短（10–13 ticks），响应最好
- FCFS：交互型响应最差（就绪 41，周转 263）

- RR：响应中等（不识别优先级但保证公平轮转）

4.3 测试3

调度算法	优先级组	就绪时间	运行时间	周转时间
DEFAULT	高优平均	2	39	41
DEFAULT	中优平均	26	18	45
DEFAULT	低优平均	51	19	70
RR	高优平均	22	87	109
RR	中优平均	19	10	30
RR	低优平均	21	11	32
PRIORITY	高优平均	2	34	36
PRIORITY	中优平均	29	31	60
PRIORITY	低优平均	55	7	62
FCFS	高优平均	6	36	42
FCFS	中优平均	28	20	49
FCFS	低优平均	68	32	100
SML	高优平均	1	30	31
SML	中优平均	28	23	51
SML	低优平均	58	15	73

- RR：三个优先级的就绪时间几乎相当，最公平
- PRIORITY / SML：高优先级响应快（周转 31–36），但低优先级等待长（就绪 55–58）
- FCFS：低优周转 100 ticks，饥饿最严重

五、结果分析

5.1 各调度算法特性分析

(1) DEFAULT (随机调度)

分析:

- 场景 1 (车队效应)
 - 长作业就绪时间极短 (1–4 ticks)，因为最先创建且在进程表前端
 - 短作业就绪 16 ticks，必须等长作业执行后才能获得 CPU
 - 随机调度倾向于选择进程表前端的进程
- 场景 2 (响应性)
 - 交互型就绪 54 ticks，响应最差
 - 完全忽略优先级，高优先级的交互任务无法优先执行
 - CPU 背景最先创建，优先获得 CPU
- 场景 3 (饥饿)
 - 表现出“弱饥饿” (低优就绪 51)，但这是创建顺序导致的
 - 随机调度不识别优先级，无法真正体现优先级差异

总结: 行为不可预测，不适合生产环境。

(2) RR (轮转调度)

分析:

- 场景 1 (车队效应)
 - 短作业周转仅 4 ticks (最优)，就绪等待 2 ticks
 - 轮转机制让短作业能快速穿插执行，无需等长作业完成
 - 长作业周转 42 ticks (较长)，因为必须与其他进程轮流执行
- 场景 2 (响应性)
 - 交互型就绪 24 ticks (中等)，因为 RR 不识别优先级
 - 虽然轮转保证公平，但无法优先调度高优先级任务
 - 对需要快速响应的交互场景不够理想
- 场景 3 (饥饿)
 - 中优和低优周转几乎相等 (30 vs 32)，完全公平
 - 高优周转 109 ticks (最长)，因为工作量大且必须轮流执行
 - 无饥饿: 所有进程公平获得 CPU 时间

总结: 最公平的算法，但牺牲了优先级响应能力。

(3) PRIORITY (优先级调度)

分析:

- 场景 1 (车队效应)
 - 所有进程优先级相同 (10)，退化为类似 FCFS 的行为

- 短作业就绪 16 ticks，车队效应明显
 - 无法展现优先级调度的优势
- 场景 2（响应性）
 - 交互型就绪 13 ticks，周转 213 ticks（最优）
 - 高优先级（3）的交互任务被优先调度
 - 这正是 **PRIORITY** 的设计目标：快速响应高优任务
- 场景 3（饥饿）
 - 高优周转 36 ticks（很快），就绪 2 ticks
 - 低优就绪 55 ticks，必须等高优和中优执行完
 - 存在饥饿风险：若持续有高优任务，低优可能永远无法执行

总结： 高优响应快，但可能导致低优饥饿。

（4）FCFS（先来先服务）

分析：

- 场景 1（车队效应）
 - 短作业周转 8 ticks（中等），到达时间错开产生插队效果
 - 长作业被其他先到达的任务阻塞，周转差异大
 - 严格按创建顺序，后创建任务必须等待
- 场景 2（响应性）
 - 交互型就绪 41 ticks，周转 263 ticks（最差）
 - 完全忽略优先级，CPU 背景先创建先执行
 - 对后到达的交互任务非常不友好
- 场景 3（饥饿）
 - 低优周转 100 ticks（所有算法中最长）
 - 低优就绪 68 ticks，必须等所有先创建的任务
 - “创建顺序饥饿”：后创建任务永远被先创建任务阻塞

总结： 严重的车队效应，对后到达任务不公平。

（5）SML（静态多级队列）

分析：

- 场景 1（车队效应）
 - 所有进程优先级 10（中层），退化为 RR 行为
 - 短作业周转 4 ticks，与 RR 相同
 - 同层轮转避免车队效应
- 场景 2（响应性）
 - 交互型就绪 10 ticks，周转 213 ticks（与 **PRIORITY** 并列最优）
 - 高优先级任务在高层队列，优先被调度
 - CPU 背景在低层，交互型 sleep 时它也能执行

- 场景 3（饥饿）
 - 高优周转 31 ticks（最快），因为独占高层队列
 - 低优就绪 58 ticks，存在跨层饥饿
 - 同层内轮转保证了层内公平

总结：兼顾优先级响应和同层公平，是最均衡的选择。

5.2 核心问题分析

（1）车队效应（Convoy Effect）

定义：短作业被长作业阻塞，导致短作业周转时间远超其实际运行时间。

表现：

- FCFS / PRIORITY / DEFAULT：短作业就绪等待长（16 ticks）
- RR / SML：短作业就绪等待短（2 ticks）

原因分析：

- FCFS 按创建顺序，长作业先到则独占 CPU
- PRIORITY 在同优先级时无法区分，表现类似 FCFS
- RR / SML 通过时间片轮转，让短作业能穿插执行

解决方案：

- 使用 RR 或 SML 调度
- 或使用最短作业优先（SJF）算法

（2）响应性（Responsiveness）

定义：交互型任务获得 CPU 的速度。

表现：

- PRIORITY / SML：交互就绪 10–13 ticks（最优）
- RR：交互就绪 24 ticks（中等）
- FCFS：交互就绪 41 ticks（最差）

原因分析：

- PRIORITY / SML 识别优先级，高优任务优先调度
- RR 公平轮转，不区分优先级

- FCFS 只看创建时间，后到达的交互任务必须等待

解决方案:

- 交互系统应使用 PRIORITY 或 SML
- 为交互任务设置高优先级

(3) 饥饿 (Starvation)

定义: 低优先级任务长时间无法获得 CPU。

表现:

- RR: 低优就绪 21 ticks, 无饥饿
- PRIORITY / SML: 低优就绪 55–58 ticks, 轻度饥饿
- FCFS: 低优就绪 68 ticks, 严重饥饿

原因分析:

- RR 不识别优先级, 所有进程公平执行
- PRIORITY 严格按优先级, 高优不结束则低优无法执行
- FCFS 按创建顺序, 后创建任务被阻塞

解决方案:

- 需要公平性时使用 RR
- 使用动态优先级 (老化机制) 避免饥饿
- SML 的同层轮转可部分缓解饥饿

5.3 算法对比总结

评价维度	DEFAULT	RR	PRIORITY	FCFS	SML
短作业响应	差	优秀	差	中等	优秀
交互响应	差	中等	优秀	差	优秀
公平性	差	优秀	中等	差	良好
避免饥饿	中等	优秀	差	差	良好
实现复杂度	简单	简单	简单	简单	中等

六、实验总结

6.1 实验收获

1. 深入理解调度算法原理

通过亲手实现五种调度算法，深刻理解了它们的选择逻辑和工作机制。特别是对轮转调度的 lastIndex 状态记录、多级队列的分层策略有了直观的认识。

2. 掌握操作系统内核编程

在 xv6 内核中修改 proc.c，学会了如何在内核态添加新的调度策略，理解了调度器与进程状态管理的关系。

3. 学会设计测试场景

针对车队效应、响应性、饥饿三个核心问题，设计了三个典型场景，能够有效区分不同调度算法的特性差异。

4. 理解调度算法的权衡

没有“最好”的调度算法，只有“最适合”的调度算法。不同场景对公平性、响应性、吞吐量有不同的需求，需要根据实际情况选择。

6.2 各算法适用场景

应用场景	推荐算法
通用服务器	SML（兼顾优先级和公平性）
交互式系统（桌面、终端）	PRIORITY 或 SML（快速响应用户输入）
批处理系统	RR（公平分配，无饥饿）
实时系统	PRIORITY（严格优先级保证）
简单嵌入式系统	FCFS（实现简单，适合单一任务流）

6.3 改进方向

1. 动态优先级调整

实现老化（Aging）机制，随时间提升等待进程的优先级，彻底避免饥饿。

2. 多级反馈队列（MLFQ）

根据进程行为动态调整所在队列层级，让 IO 密集型进程自动获得更高优先级。

3. 完全公平调度（CFS）

使用虚拟运行时间，保证每个进程获得公平的 CPU 份额，这是 Linux 的默认调度器。

4. 实时调度支持

实现 EDF（最早截止时间优先）或 RMS（速率单调调度），支持硬实时任务。

6.4 实验结论

本实验成功实现了五种经典调度算法，并通过三个精心设计的测试场景验证了它们的性能特点。实验结果表明：

1. RR 是最公平的算法，能完全避免饥饿，但无法识别优先级。
2. PRIORITY 响应性最好，高优任务能快速执行，但可能导致低优任务饥饿。
3. FCFS 实现最简单，但存在严重的车队效应，对后到达任务不友好。
4. SML 是最均衡的选择，结合了优先级调度的响应性和轮转调度的公平性，适合大多数通用场景。
5. DEFAULT（随机调度）行为不可预测，仅适合测试和调试，不建议生产使用。

调度算法的选择应根据具体应用场景的需求来决定。理解各算法的优缺点，是操作系统设计和优化的重要基础。

附录

1 测试结果与测试数据

场景1：纯CPU长短作业（车队效应）

DEFAULT - 场景1

项	类型	就绪时间	运行时间	周转时间
4	长作业	1	22	23
5	长作业	4	23	27
6	短作业	16	1	17
7	短作业	16	2	18
8	中作业	10	5	15
9	中作业	12	7	19
长作业平均	长作业	2	22	25
短作业平均	短作业	16	1	17
中作业平均	中作业	11	6	17

RR(轮转) - 场景1

项	类型	就绪时间	运行时间	周转时间
22	长作业	7	24	31
23	长作业	7	47	106
24	短作业	2	1	3
25	短作业	3	3	6
26	中作业	8	8	27
27	中作业	6	3	12
长作业平均	长作业	7	35	42
短作业平均	短作业	2	2	4
中作业平均	中作业	7	5	12

PRIORITY - 场景1

项	类型	就绪时间	运行时间	周转时间
40	长作业	0	17	17
41	长作业	2	28	30
42	短作业	12	3	20
43	短作业	21	1	27
44	中作业	12	5	17
45	中作业	16	8	36
长作业平均	长作业	1	22	23
短作业平均	短作业	16	2	18
中作业平均	中作业	14	6	20

FCFS - 场景1

项	类型	就绪时间	运行时间	周转时间
59	长作业	0	17	17
58	长作业	2	34	72
60	短作业	7	1	8
61	短作业	8	1	9
62	中作业	4	35	52
63	中作业	17	7	52
长作业平均	长作业	1	25	26

项	类型	就绪时间	运行时间	周转时间
短作业平均	短作业	7	1	8
中作业平均	中作业	10	21	31

SML(多级队列) - 场景1

项	类型	就绪时间	运行时间	周转时间
76	长作业	2	18	20
77	长作业	7	34	76
78	短作业	2	1	3
79	短作业	3	3	6
80	中作业	3	3	8
81	中作业	3	5	8
长作业平均	长作业	4	26	30
短作业平均	短作业	2	2	4
中作业平均	中作业	3	4	7

场景2：交互型 vs 背景任务

DEFAULT - 场景2

项	类型	就绪时间	运行时间	休眠时间	周转时间
10	CPU 背景	2	23	7	32
11	CPU 背景	4	31	0	35
13	交互型	54	2	169	225
12	交互型	55	10	208	273
15	IO 背景	75	34	410	519
14	IO 背景	73	28	535	636
交互型平均	交互型	54	6	188	249
CPU 背景平均	CPU 背景	3	27	0	30
IO 背景平均	IO 背景	74	31	472	577

RR(轮转) - 场景2

项	类型	就绪时间	运行时间	休眠时间	周转时间
29	CPU 背景	9	23	8	40
28	CPU 背景	11	39	61	111
31	交互型	25	2	181	208
30	交互型	24	5	246	275
33	IO 背景	45	40	548	633
32	IO 背景	43	42	610	695
交互型平均	交互型	24	3	213	241
CPU 背景平均	CPU 背景	10	31	0	41
IO 背景平均	IO 背景	44	41	579	664

PRIORITY - 场景2

项	类型	就绪时间	运行时间	休眠时间	周转时间
46	CPU 背景	1	26	0	27
47	CPU 背景	7	24	58	89
48	交互型	13	5	195	213
49	交互型	14	4	195	213
50	IO 背景	33	21	602	656
51	IO 背景	45	26	681	752
交互型平均	交互型	13	4	195	213
CPU 背景平均	CPU 背景	4	25	0	29
IO 背景平均	IO 背景	39	23	641	704

FCFS - 场景2

项	类型	就绪时间	运行时间	休眠时间	周转时间
64	CPU 背景	3	20	0	23
65	CPU 背景	6	33	23	62
67	交互型	43	9	182	234
66	交互型	40	32	221	293
68	IO 背景	77	25	484	586

项	类型	就绪时间	运行时间	休眠时间	周转时间
69	IO 背景	88	32	521	641
交互型平均	交互型	41	20	201	263
CPU 背景平均	CPU 背景	4	26	0	31
IO 背景平均	IO 背景	82	28	502	613

SML(多级队列) - 场景2

项	类型	就绪时间	运行时间	休眠时间	周转时间
82	CPU 背景	15	18	0	33
83	CPU 背景	18	18	10	46
84	交互型	10	5	198	213
85	交互型	10	1	202	213
86	IO 背景	26	67	542	635
87	IO 背景	37	45	641	723
交互型平均	交互型	10	3	200	213
CPU 背景平均	CPU 背景	16	18	0	34
IO 背景平均	IO 背景	31	56	591	679

场景3：优先级饥饿测试

DEFAULT - 场景3

项	优先级组	就绪时间	运行时间	周转时间
17	中优先级	5	27	32
16	高优先级	2	39	41
18	中优先级	30	13	43
19	中优先级	44	16	79
20	低优先级	47	10	79
21	低优先级	56	28	90

项	优先级组	就绪时间	运行时间	周转时间
高优平均	高优先级	2	39	41
中优平均	中优先级	26	18	45
低优平均	低优先级	51	19	70

RR(轮转) - 场景3

项	优先级组	就绪时间	运行时间	周转时间
35	中优先级	16	8	35
34	高优先级	22	87	141
36	中优先级	22	13	35
37	中优先级	21	10	35
38	低优先级	22	15	37
39	低优先级	20	8	35
高优平均	高优先级	22	87	109
中优平均	中优先级	19	10	30
低优平均	低优先级	21	11	32

PRIORITY - 场景3

项	优先级组	就绪时间	运行时间	周转时间
52	高优先级	2	34	36
53	中优先级	10	28	91
54	中优先级	38	6	91
55	中优先级	41	59	102
56	低优先级	52	8	102
57	低优先级	58	6	102
高优平均	高优先级	2	34	36
中优平均	中优先级	29	31	60
低优平均	低优先级	55	7	62

FCFS - 场景3

项	优先级组	就绪时间	运行时间	周转时间
71	中优先级	1	21	22
70	高优先级	6	36	64
72	中优先级	34	30	122
73	中优先级	50	11	122
74	低优先级	64	14	122
75	低优先级	72	50	122
高优平均	高优先级	6	36	42
中优平均	中优先级	28	20	49
低优平均	低优先级	68	32	100

SML(多级队列) - 场景3

项	优先级组	就绪时间	运行时间	周转时间
88	高优先级	1	30	31
89	中优先级	29	30	74
90	中优先级	32	18	73
91	中优先级	25	21	73
92	低优先级	58	11	73
93	低优先级	59	19	81
高优平均	高优先级	1	30	31
中优平均	中优先级	28	23	51
低优平均	低优先级	58	15	73

2 关键代码

```
// The following funtions are used by scheduler() for selecting
// the next process to be executed.
// All of them select a process (using a different policy) among runnables.
```

```

////////////////////////////////////
// Scheduler policies - Pre-installed code //
////////////////////////////////////

// Default scheduler -----
struct proc *defaultScheduler(void) {
    struct proc *p;
    int i, rnd;

    rnd = random(NPROC);
    for (i = 0; i < NPROC; i++) {
        p = ptable.proc + ((rnd + i) % NPROC);
        if (p->state == RUNNABLE) return p;
    }
    return 0;
}

// Priority Scheduler -----
// 选择优先级值最小（优先级最高）的 RUNNABLE 进程
struct proc *priorityScheduler() {
    struct proc *p;
    struct proc *highP = 0;

    // 遍历进程表，找到优先级最高的进程
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++) {
        if(p->state != RUNNABLE)
            continue;
        // 如果还没找到或当前进程优先级更高（值更小）
        if(highP == 0 || p->priority < highP->priority) {
            highP = p;
        }
    }
    return highP;
}

// FCFS Scheduler -----
// 先来先服务：选择创建时间最早的 RUNNABLE 进程
struct proc *fcfsScheduler() {
    struct proc *p;
    struct proc *earliest = 0;

    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++) {
        if(p->state != RUNNABLE)
            continue;
        // 选择 ctime 最小（最早创建）的进程
        if(earliest == 0 || p->ctime < earliest->ctime) {
            earliest = p;
        }
    }
    return earliest;
}

// RR Scheduler (Round Robin) -----

```



```

// 轮转调度：循环遍历，轮流选择进程
struct proc *rrScheduler() {
    struct proc *p;
    static int lastIdx = 0; // 记录上次调度的位置
    int i;

    // 从上次位置的下一个开始查找
    for(i = 0; i < NPROC; i++) {
        int idx = (lastIdx + i) % NPROC;
        p = &ptable.proc[idx];
        if(p->state == RUNNABLE) {
            lastIdx = (idx + 1) % NPROC; // 更新下次起始位置
            return p;
        }
    }
    return 0;
}

// SML Scheduler (Static Multi-Level Queue)
// 静态多级队列：按优先级分 3 层
// 第 1 层: priority 1-6 (高优先级)
// 第 2 层: priority 7-13 (中优先级)
// 第 3 层: priority 14-20 (低优先级)
// 高层队列有进程时优先执行，同层内轮转
struct proc *smlScheduler() {
    struct proc *p;
    static int lastIdx[3] = {0, 0, 0}; // 每层队列的轮转位置
    int i, level;

    // 按优先级从高到低遍历 3 个队列
    for(level = 0; level < 3; level++) {
        for(i = 0; i < NPROC; i++) {
            int idx = (lastIdx[level] + i) % NPROC;
            p = &ptable.proc[idx];

            if(p->state != RUNNABLE)
                continue;

            // 判断进程属于哪个队列
            int pLevel;
            if(p->priority >= 1 && p->priority <= 6)
                pLevel = 0; // 高优先级队列
            else if(p->priority >= 7 && p->priority <= 13)
                pLevel = 1; // 中优先级队列
            else
                pLevel = 2; // 低优先级队列

            // 只选择当前层的进程
            if(pLevel == level) {
                lastIdx[level] = (idx + 1) % NPROC;
                return p;
            }
        }
    }
}

```

```

}
return 0;
}

```

```

// 调度算法测试程序
// 三个典型场景：车队效应、交互型 vs 背景、优先级饥饿
// 对比不同调度算法的性能指标，结果同时输出到屏幕和文件

```

```

#include "types.h"
#include "user.h"
#include "fcntl.h"

```

```

#define NCHILD 6      // 子进程数量
#define CPU_WORK 500000 // CPU 基准工作量
#define IO_WORK 20    // IO 基准次数

```

```

// 调度算法编号（与内核保持一致）

```

```

#define SCHED_DEFAULT 0
#define SCHED_PRIORITY 1
#define SCHED_FCFS 2
#define SCHED_RR 3
#define SCHED_SML 4

```

```

// 任务类型

```

```

#define ROLE_LONG 0 // 长任务
#define ROLE_SHORT 1 // 短任务
#define ROLE_MEDIUM 2 // 中等任务
#define ROLE_INTER 3 // 交互型
#define ROLE_IO_BG 4 // 后台 IO
#define ROLE_CPU_BG 5 // 后台 CPU
#define ROLE_HIGH 6 // 高优先级
#define ROLE_MID 7 // 中优先级
#define ROLE_LOW 8 // 低优先级

```

```

// 全局文件描述符，用于输出到文件

```

```

int outfd = -1;

```

```

struct child_info {
    int pid;
    int role;
};

```

```

// 同时输出到屏幕和文件

```

```

void output(char *s) {
    printf(1, "%s", s);
    if(outfd >= 0) {
        write(outfd, s, strlen(s));
    }
}

```

```

}
}

// 输出数字
void output_int(int n) {
    char buf[16];
    int i = 0, neg = 0;
    if(n < 0) { neg = 1; n = -n; }
    do { buf[i++] = '0' + (n % 10); n /= 10; } while(n > 0);
    if(neg) buf[i++] = '-';
    char out[16];
    int j;
    for(j = 0; j < i; j++) out[j] = buf[i-1-j];
    out[j] = 0;
    output(out);
}

// CPU 密集型工作
void cpu_work(int work) {
    int i, j;
    volatile int sum = 0;
    for(i = 0; i < work; i++) {
        for(j = 0; j < 100; j++) {
            int x = i * j + (i ^ j);
            x = (x << 1) ^ (x >> 1);
            sum += x;
        }
    }
}

// IO 密集型工作
void io_work(int rounds) {
    int i, fd;
    char buf[] = "io test\n";
    for(i = 0; i < rounds; i++) {
        fd = open("io_test", O_CREATE | O_WRONLY);
        if(fd >= 0) {
            write(fd, buf, sizeof(buf) - 1);
            close(fd);
        }
        sleep(15); // 模拟 IO 等待
    }
}

// 交互型工作：短 CPU + 频繁 sleep（模拟用户输入等待）
void interactive_work(int rounds) {
    int i;
    for(i = 0; i < rounds; i++) {
        cpu_work(CPU_WORK / 50); // 很短的 CPU 计算
        sleep(10);                // 等待用户/IO
    }
}

```

```

// =====
// 场景 1: 纯 CPU 长短作业 - 展示 FCFS 车队效应 & RR 公平性
// =====
void test_scenario1(int schedId, char *schedName) {
    int pid, i, k;
    int retime, ruptime, stime;
    struct child_info info[NCHILD];
    int nchildren = 0;

    // 统计变量
    int long_retime = 0, long_ruptime = 0, long_count = 0;
    int short_retime = 0, short_ruptime = 0, short_count = 0;
    int med_retime = 0, med_ruptime = 0, med_count = 0;

    int cpu_long = CPU_WORK;
    int cpu_short = CPU_WORK / 10;
    int cpu_medium = CPU_WORK / 3;

    output("\n===== ");
    output(schedName);
    output(" - 场景 1: 纯 CPU 长短作业(车队效应) =====\n");

    setscheduler(schedId);

    // P1: 长作业, 到达时间 0
    pid = fork();
    if(pid == 0) {
        setpriority(getpid(), 10);
        cpu_work(cpu_long);
        exit();
    }
    if(pid > 0) { info[nchildren].pid = pid; info[nchildren].role = ROLE_LONG; nchildren++; }

    // P2: 长作业, 到达时间 0
    pid = fork();
    if(pid == 0) {
        setpriority(getpid(), 10);
        cpu_work(cpu_long);
        exit();
    }
    if(pid > 0) { info[nchildren].pid = pid; info[nchildren].role = ROLE_LONG; nchildren++; }

    sleep(10); // 等一会儿再创建短作业

    // P3: 短作业, 稍晚到达
    pid = fork();
    if(pid == 0) {
        setpriority(getpid(), 10);
        cpu_work(cpu_short);
        exit();
    }
    if(pid > 0) { info[nchildren].pid = pid; info[nchildren].role = ROLE_SHORT; nchildren++; }
}

```

```
// P4: 短作业
pid = fork();
if(pid == 0) {
    setpriority(getpid(), 10);
    cpu_work(cpu_short);
    exit();
}
if(pid > 0) { info[nchildren].pid = pid; info[nchildren].role = ROLE_SHORT; nchildren++; }

sleep(10);

// P5: 中等作业
pid = fork();
if(pid == 0) {
    setpriority(getpid(), 10);
    cpu_work(cpu_medium);
    exit();
}
if(pid > 0) { info[nchildren].pid = pid; info[nchildren].role = ROLE_MEDIUM; nchildren++; }

// P6: 中等作业
pid = fork();
if(pid == 0) {
    setpriority(getpid(), 10);
    cpu_work(cpu_medium);
    exit();
}
if(pid > 0) { info[nchildren].pid = pid; info[nchildren].role = ROLE_MEDIUM; nchildren++; }

// 收集统计
output("PID\t类型\t就绪时间\t运行时间\t周转时间\n");
output("----\t---\t\t\t-----\t-----\t-----\n");

for(i = 0; i < nchildren; i++) {
    pid = wait2(&retime, &rutime, &stime);
    if(pid > 0) {
        int turnaround = retime + rutime + stime;
        int role = ROLE_MEDIUM;
        for(k = 0; k < nchildren; k++) {
            if(info[k].pid == pid) { role = info[k].role; break; }
        }

        char *type = (role == ROLE_LONG) ? "长作业" :
            (role == ROLE_SHORT) ? "短作业" : "中作业";

        output_int(pid); output("\t"); output(type); output("\t\t");
        output_int(retime); output("\t\t");
        output_int(rutime); output("\t\t");
        output_int(turnaround); output("\n");

        if(role == ROLE_LONG) {
            long_retme += retime; long_rutime += rutime; long_count++;
        } else if(role == ROLE_SHORT) {
```

```

    short_retime += retime; short_rutime += rutime; short_count++;
} else {
    med_retime += retime; med_rutime += rutime; med_count++;
}
}
}

output("----\t----\t\t-----\t-----\t-----\n");
if(long_count > 0) {
    output("长作业平均\t\t"); output_int(long_retime/long_count);
    output("\t\t"); output_int(long_rutime/long_count);
    output("\t\t"); output_int((long_retime+long_rutime)/long_count); output("\n");
}
if(short_count > 0) {
    output("短作业平均\t\t"); output_int(short_retime/short_count);
    output("\t\t"); output_int(short_rutime/short_count);
    output("\t\t"); output_int((short_retime+short_rutime)/short_count); output("\n");
}
if(med_count > 0) {
    output("中作业平均\t\t"); output_int(med_retime/med_count);
    output("\t\t"); output_int(med_rutime/med_count);
    output("\t\t"); output_int((med_retime+med_rutime)/med_count); output("\n");
}
}

// =====
// 场景 2: 交互型 vs 背景任务 - 展示 RR/SML 对交互友好
// =====

void test_scenario2(int schedId, char *schedName) {
    int pid, i, k;
    int retime, rutime, stime;
    struct child_info info[NCHILD];
    int nchildren = 0;

    int inter_retime = 0, inter_rutime = 0, inter_stime = 0, inter_count = 0;
    int cpubg_retime = 0, cpubg_rutime = 0, cpubg_count = 0;
    int iobg_retime = 0, iobg_rutime = 0, iobg_stime = 0, iobg_count = 0;

    output("\n===== ");
    output(schedName);
    output("- 场景 2: 交互型 vs 背景任务 =====\n");

    setscheduler(schedId);

    // 2 个重 CPU 背景任务 (低优先级)
    for(i = 0; i < 2; i++) {
        pid = fork();
        if(pid == 0) {
            setpriority(getpid(), 18); // 低优先级
            cpu_work(CPU_WORK);
            exit();
        }
        if(pid > 0) { info[nchildren].pid = pid; info[nchildren].role = ROLE_CPU_BG; nchildren++; }
    }
}

```

```

}

// 2 个交互型任务（高优先级，短 CPU+频繁 IO）
for(i = 0; i < 2; i++) {
    pid = fork();
    if(pid == 0) {
        setpriority(getpid(), 3); // 高优先级
        interactive_work(IO_WORK);
        exit();
    }
    if(pid > 0) { info[nchildren].pid = pid; info[nchildren].role = ROLE_INTER; nchildren++; }
}

// 2 个后台 IO 任务（中等优先级）
for(i = 0; i < 2; i++) {
    pid = fork();
    if(pid == 0) {
        setpriority(getpid(), 12);
        io_work(IO_WORK);
        exit();
    }
    if(pid > 0) { info[nchildren].pid = pid; info[nchildren].role = ROLE_IO_BG; nchildren++; }
}

output("PID\t 类型\t\t 就绪时间\t 运行时间\t 休眠时间\t 周转时间\n");
output("-----\t-----\t-----\t-----\t-----\t-----\n");

for(i = 0; i < nchildren; i++) {
    pid = wait2(&retime, &rutime, &stime);
    if(pid > 0) {
        int turnaround = retime + rutime + stime;
        int role = ROLE_CPU_BG;
        for(k = 0; k < nchildren; k++) {
            if(info[k].pid == pid) { role = info[k].role; break; }
        }

        char *type = (role == ROLE_INTER) ? "交互型" :
            (role == ROLE_CPU_BG) ? "CPU 背景" : "IO 背景";

        output_int(pid); output("\t"); output(type); output("\t\t");
        output_int(retime); output("\t\t");
        output_int(rutime); output("\t\t");
        output_int(stime); output("\t\t");
        output_int(turnaround); output("\n");

        if(role == ROLE_INTER) {
            inter_retime += retime; inter_rutime += rutime; inter_stime += stime; inter_count++;
        } else if(role == ROLE_CPU_BG) {
            cpubg_retime += retime; cpubg_rutime += rutime; cpubg_count++;
        } else {
            iobg_retime += retime; iobg_rutime += rutime; iobg_stime += stime; iobg_count++;
        }
    }
}

```

```

}

output("----\t---\t\t-----\t-----\t-----\t-----\n");
if(inter_count > 0) {
    output("交互型平均\t\t"); output_int(inter_retime/inter_count);
    output("\t\t"); output_int(inter_rutime/inter_count);
    output("\t\t"); output_int(inter_stime/inter_count);
    output("\t\t"); output_int((inter_retime+inter_rutime+inter_stime)/inter_count); output("\n");
}
if(cpupg_count > 0) {
    output("CPU 背景平均\t\t"); output_int(cpupg_retime/cpupg_count);
    output("\t\t"); output_int(cpupg_rutime/cpupg_count);
    output("\t\t\t\t");
    output_int((cpupg_retime+cpupg_rutime)/cpupg_count); output("\n");
}
if(iobg_count > 0) {
    output("IO 背景平均\t\t"); output_int(iobg_retime/iobg_count);
    output("\t\t"); output_int(iobg_rutime/iobg_count);
    output("\t\t"); output_int(iobg_stime/iobg_count);
    output("\t\t"); output_int((iobg_retime+iobg_rutime+iobg_stime)/iobg_count); output("\n");
}
}

// =====
// 场景 3：优先级饥饿测试 - 展示 Priority 问题 & SML/RR 缓解
// =====

void test_scenario3(int schedId, char *schedName) {
    int pid, i, k;
    int retime, rutime, stime;
    struct child_info info[NCHILD];
    int nchildren = 0;

    int high_retime = 0, high_rutime = 0, high_count = 0;
    int mid_retime = 0, mid_rutime = 0, mid_count = 0;
    int low_retime = 0, low_rutime = 0, low_count = 0;

    int cpu_verylong = CPU_WORK * 2;
    int cpu_medium = CPU_WORK / 2;

    output("\n===== ");
    output(schedName);
    output(" - 场景 3: 优先级饥饿测试 =====\n");

    setscheduler(schedId);

    // P1: 超高优先级 CPU 大作业（会霸占 CPU）
    pid = fork();
    if(pid == 0) {
        setpriority(getpid(), 1); // 最高优先级
        cpu_work(cpu_verylong);
        exit();
    }
    if(pid > 0) { info[nchildren].pid = pid; info[nchildren].role = ROLE_HIGH; nchildren++; }
}

```



```

// P2-P4: 中等优先级小任务
for(i = 0; i < 3; i++) {
    pid = fork();
    if(pid == 0) {
        setpriority(getpid(), 10); // 中优先级
        cpu_work(cpu_medium);
        exit();
    }
    if(pid > 0) { info[nchildren].pid = pid; info[nchildren].role = ROLE_MID; nchildren++; }
}

// P5-P6: 最低优先级小任务（可能饥饿）
for(i = 0; i < 2; i++) {
    pid = fork();
    if(pid == 0) {
        setpriority(getpid(), 19); // 最低优先级
        cpu_work(cpu_medium);
        exit();
    }
    if(pid > 0) { info[nchildren].pid = pid; info[nchildren].role = ROLE_LOW; nchildren++; }
}

output("PID\t 优先级组\t 就绪时间\t 运行时间\t 周转时间\n");
output("----\t-----\t-----\t-----\t-----\n");

for(i = 0; i < nchildren; i++) {
    pid = wait2(&retime, &rtime, &stime);
    if(pid > 0) {
        int turnaround = retime + rtime + stime;
        int role = ROLE_MID;
        for(k = 0; k < nchildren; k++) {
            if(info[k].pid == pid) { role = info[k].role; break; }
        }

        char *type = (role == ROLE_HIGH) ? "高优先级" :
            (role == ROLE_MID) ? "中优先级" : "低优先级";

        output_int(pid); output("\t"); output(type); output("\t");
        output_int(retime); output("\t");
        output_int(rtime); output("\t");
        output_int(turnaround); output("\n");

        if(role == ROLE_HIGH) {
            high_retime += retime; high_rtime += rtime; high_count++;
        } else if(role == ROLE_MID) {
            mid_retime += retime; mid_rtime += rtime; mid_count++;
        } else {
            low_retime += retime; low_rtime += rtime; low_count++;
        }
    }
}

```

```

output("----\t-----\t-----\t-----\t-----\n");
if(high_count > 0) {
    output("高优平均\t\t"); output_int(high_retime/high_count);
    output("\t\t"); output_int(high_rutime/high_count);
    output("\t\t"); output_int((high_retime+high_rutime)/high_count); output("\n");
}
if(mid_count > 0) {
    output("中优平均\t\t"); output_int(mid_retime/mid_count);
    output("\t\t"); output_int(mid_rutime/mid_count);
    output("\t\t"); output_int((mid_retime+mid_rutime)/mid_count); output("\n");
}
if(low_count > 0) {
    output("低优平均\t\t"); output_int(low_retime/low_count);
    output("\t\t"); output_int(low_rutime/low_count);
    output("\t\t"); output_int((low_retime+low_rutime)/low_count); output("\n");
}
}

// =====
// 主函数
// =====
int main(int argc, char *argv[]) {
    // 打开输出文件
    outfd = open("sched_result.txt", O_CREATE | O_WRONLY);

    output("=====\n");
    output("    调度算法性能比较测试 (3 场景)    \n");
    output("=====\n");
    output("场景 1: 纯 CPU 长短作业 - 测试车队效应\n");
    output("场景 2: 交互型 vs 背景 - 测试响应性\n");
    output("场景 3: 优先级饥饿 - 测试公平性\n");
    output("=====\n");

    // 调度算法列表
    int sched_ids[] = { SCHED_DEFAULT, SCHED_RR, SCHED_PRIORITY, SCHED_FCFS, SCHED_SML };
    char *sched_names[] = {
        "DEFAULT",
        "RR(轮转)",
        "PRIORITY",
        "FCFS",
        "SML(多级队列)"
    };

    int nsched = 5;

    int si;
    for(si = 0; si < nsched; si++) {
        // 场景 1
        test_scenario1(sched_ids[si], sched_names[si]);
        // 场景 2
        test_scenario2(sched_ids[si], sched_names[si]);
        // 场景 3
        test_scenario3(sched_ids[si], sched_names[si]);
    }
}

```

```
output("\n===== 测试完成 =====\n");
output("结果已保存到 sched_result.txt\n");

if(outfd >= 0) close(outfd);

// 恢复默认调度器
setscheduler(SCHED_DEFAULT);

exit();
}
```